SUPPORTHUB

Complete Project Blueprint

A build-first guide for planning and developing a customer support ticket management system with Express.js, followed by a React + Vite frontend.

Project focus	A practical customer support platform for creating, assigning, tracking and resolving customer requests.
Backend	Node.js and Express.js first, using temporary in-memory data before introducing MongoDB/Mongoose.
Frontend	React + Vite after the backend API and its permissions are working.
Learning approach	Research concepts just in time, implement one feature at a time, and review the code as the project grows.

How to use this document

Treat this blueprint as the project's reference specification. It explains what you are building, why each part exists, the expected behaviour, the planned files and API, and the milestones. You do not need to create every file immediately. Follow the roadmap in order and only add structure when a feature needs it.

1. Project overview

SupportHub is a customer support ticket management system. It gives a company one place to receive customer problems or requests, organize them, assign them to support staff, communicate with customers and track each request until it is resolved.

The project is designed to help you practise backend development with Express.js in a realistic context. You will build the API first, beginning with a small working server and gradually adding users, tickets, comments, authentication, permissions, validation, tests and persistent storage.

The first version uses temporary JavaScript arrays for data so you can focus on HTTP, routes, controllers, services and business rules. MongoDB and Mongoose come later, after the API behaviour is understood.

2. The problem SupportHub solves

Imagine a company called TechWorld that sells computers and provides technical support. Customers contact the company with different issues: they cannot sign in, a device is not working, an order has not arrived, or they need help using a product.

Without a support system, requests can get lost in messages or become difficult to track. Staff may not know who is handling a request, customers may not know whether anyone has responded, and managers may struggle to see unresolved work.

SupportHub organizes each request as a ticket. A ticket has an owner, a status, a priority, a conversation history and optionally an assigned support agent. This makes the work visible and trackable.

What the application does — and does not do

SupportHub records and manages the support process. It does not automatically repair a customer's laptop, deliver an order or perform the real-world service itself. It helps the customer and support team communicate and keep track of what needs to happen.

3. Core terminology: what is a ticket?

A **support ticket** is a stored record representing one customer's problem, question or request. Think of it as a trackable case that the company can work on until it is resolved.

Example: a customer cannot connect their laptop to Wi-Fi. They sign in to SupportHub and create a ticket with a short title and a description of the problem. The system stores the request, gives it an ID, sets its initial status to *open* and links it to that customer.

A support agent can then review the ticket, ask questions, suggest next steps and update its status. The customer can return to the same ticket to read the response and provide more information. The conversation remains attached to the ticket.

Other examples include account access problems, billing questions, delivery issues, product questions and software bugs. Each separate request should normally have its own ticket.

Important terms

• **Customer:** the person who needs help and creates a request.
• **Support agent:** a staff member who handles customer requests.
• **Admin:** a person who manages the platform and its staff access.
• **Assignment:** the act of allocating a ticket to an agent.
• **Status:** the current stage of a ticket.
• **Comment:** a message added to a ticket's conversation.

4. Main product experience

The first version has three role-specific experiences:

• **Customer:** register, sign in, create a ticket, see their own tickets, open a ticket and reply to its conversation.
• **Support agent:** sign in, view the tickets they are allowed to handle, open a ticket, reply, and update its status.
• **Admin:** manage users and support agents, view all tickets, assign tickets and oversee the system.

The backend is responsible for deciding what each role is allowed to do. The frontend will present the appropriate pages and controls, but hiding a button in the browser is not security. Every protected action must also be checked by the API.

5. How SupportHub works from start to finish

1. A customer creates an account through public registration. The account receives the customer role automatically.

2. The customer signs in and submits a support request. SupportHub creates a ticket linked to that customer and gives it an initial status of *open*.

3. The ticket appears in the support queue. An administrator or the system's agreed assignment process allocates it to a support agent.

4. The support agent reviews the request and communicates with the customer through comments on the ticket. The agent can update the status as work progresses.

5. The customer can sign in later to read the conversation and respond. The ticket stays connected to the same customer and keeps its history.

6. Once the issue has been addressed, the ticket can be marked resolved and later closed according to the workflow rules.

The three account roles

Role	Who they are	Primary responsibility
Customer	A person requesting help	Create tickets and follow/reply to their own requests
Support agent	A company staff member	Handle assigned or permitted tickets and communicate with customers
Admin	A trusted platform administrator	Manage users, agents, ticket assignments and overall operations

How accounts are created

Anyone may use public registration to become a customer. A person must not be able to select *support_agent* or *admin* in the registration form. Those accounts are created or provisioned through a controlled administrative process because those roles grant access to customer information and management functions.

The first administrator is also set up through a controlled initial setup process, not through an open public sign-up option. The detailed permission and security rules follow in section 6.

6. Role security rules

- Never trust a role sent by the browser. The backend decides the role.
- If a public registration request includes role=admin or role=support_agent, ignore or reject that field; it must not grant elevated access.
- A customer can access only their own tickets and ticket conversations.
- An agent can access tickets they are assigned to, plus any unassigned or team-wide queue explicitly allowed by the product rules.
- An admin can view and manage all tickets and user accounts.
- The frontend may hide controls a role cannot use, but the backend must enforce the same restriction on every protected endpoint.
- Do not store plain-text passwords. Password hashing is introduced in the authentication phase.
- Do not commit secrets, credentials, tokens or the .env file to GitHub.

7. Permission matrix

Action	Customer	Support agent	Admin
Register a public account	Yes	No public role selection	No public role selection
Create a ticket	Yes	Optional internal ticket later	Optional internal ticket later
View own tickets	Yes	If also customer is not in initial scope	If also customer is not in initial scope
View all tickets	No	Only permitted queue/assigned tickets	Yes
Reply to a ticket	Own tickets	Tickets they can access	For oversight/support if permitted
Change ticket status	Limited customer confirmation/reopen rules	Yes, on accessible tickets	Yes
Assign an agent	No	No, unless future team-lead feature	Yes
Create agent accounts	No	No	Yes
Manage user roles	No	No	Yes
Delete/disable accounts	No	No	Yes, subject to safeguards

Initial implementation decision: public registration creates customers only. An admin provisions agent accounts. The initial admin is seeded through a controlled setup process. This keeps role assignment simple and secure for the first version.

8. Account states

A user account should eventually have an account status such as active or disabled. A disabled account cannot sign in or use protected endpoints. This is separate from a ticket's status. Account activation and email verification can be considered after the core authentication flow works.

C. Ticket workflows and business rules

9. Ticket lifecycle

Step	What happens	System result
1. Submit	Customer signs in and submits a title, description and optionally a category/priority.	Create a ticket linked to the authenticated customer. Generate its ID and set its initial status to open.
2. Queue	The new request becomes visible to the permitted support team.	Ticket appears in the unassigned/open queue.
3. Assign	An admin assigns an agent, or an agent claims an unassigned ticket if that feature is enabled.	Store the assigned agent's user ID.
4. Respond	Agent adds a reply or asks for more information.	Store a comment linked to the ticket and its author.
5. Customer replies	Customer answers within the ticket conversation.	Store the customer comment and make the updated conversation visible to the assigned agent.
6. Work	Agent investigates and updates progress.	Status may move to in_progress or waiting_on_customer.
7. Resolve	Agent believes the issue is addressed.	Set status to resolved and record who resolved it and when.
8. Close / reopen	The request is closed after completion, or reopened if more help is needed under the chosen rules.	Update status and retain the history.

10. Ticket statuses

Status value	Meaning	Typical transition
open	New request awaiting attention.	Initial state; may be assigned or moved to in_progress.
in_progress	An agent is actively working on the request.	Can move to waiting_on_customer or resolved.
waiting_on_customer	Agent needs more information or a response from the customer.	Customer replies; agent resumes in_progress.
resolved	Support believes the request has been addressed.	May be closed or reopened if the customer needs more help.
closed	The request is finished; no further action is expected.	A new issue should normally be a new ticket. Reopening can be allowed by a defined policy.

Status rules

- A newly created ticket starts as open. The customer does not choose a final status.
- An agent may move a ticket to in_progress while working on it.
- An agent may use waiting_on_customer when a customer response is required.
- An agent marks a ticket resolved when a solution has been provided or the request has been addressed.
- For the first version, admins can override ticket status when necessary.
- A customer may add a reply to an accessible ticket. Reopening a resolved/closed ticket can be added with an explicit rule; do not assume every customer can arbitrarily set any status.
- The backend validates every status transition. A request must not be able to set an unsupported status value.

11. Ticket priorities

Priority	Meaning
low	A non-urgent request that can wait in the normal queue.
medium	A normal request requiring attention in the usual workflow.
high	A request that should be handled sooner than routine work.
urgent	A time-sensitive or severe issue requiring prompt attention.

Priority and status are different. A ticket can be high priority and still be open. Priority helps determine urgency; status describes progress.

12. Assignment rules

- A ticket may begin without an assigned agent.
- An admin can assign an active support agent to a ticket.
- The backend must verify that the target account exists, is active and has the support_agent role.
- Customers cannot assign tickets to agents.
- For the initial version, agents cannot reassign tickets to other agents. This can be added later if needed.
- When an agent is assigned, the ticket should be visible in that agent's assigned-ticket view.

13. Comments and conversations

- Each comment belongs to exactly one ticket and has an author.
- The author is derived from the authenticated user, not trusted from a client-supplied userId.
- Customers can comment only on their own tickets.
- Agents can comment only on tickets they are allowed to access.
- Comments are shown in chronological order, oldest first in the conversation view.
- For the first version, comments are plain text. File attachments, images, internal agent-only notes and rich text are future enhancements.
- A ticket comment is not a separate ticket. It is one message inside the ticket's conversation.

D. Data and record specification

14. Temporary data before MongoDB

During the early Express phases, use JavaScript arrays of objects as temporary storage. This is intentional: it lets you practise request handling and business rules before introducing database modelling. In-memory records disappear when the server restarts unless you add file persistence; do not mistake this for a production database.

15. User record — planned fields

Field	Type / example	Purpose
id	string or number	Unique user identifier in the temporary implementation.
name	"Slim Sawi"	User's display name.
email	"slim@example.com"	Unique sign-in/contact address.
passwordHash	stored hash later	Never store the plain-text password.
role	customer | support_agent | admin	Permission role assigned by trusted backend logic.
accountStatus	active | disabled	Whether the account can use the system.
createdAt	ISO timestamp	When the account was created.
updatedAt	ISO timestamp	When the account was last updated.

16. Ticket record — planned fields

Field	Type / example	Purpose
id	unique ID	Identifies the ticket.
title	"Laptop Wi-Fi issue"	Short summary.
description	Text	Customer's explanation of the issue.
customerId	user ID	The customer who submitted it.
assignedAgentId	user ID or null	Agent responsible; null while unassigned.
status	open, in_progress, etc.	Current workflow state.
priority	low, medium, high, urgent	Urgency.
category	technical_support, billing, etc.	Optional classification.
createdAt	ISO timestamp	Submission time.
updatedAt	ISO timestamp	Last update time.
resolvedAt	timestamp or null	When it was resolved, if applicable.
closedAt	timestamp or null	When it was closed, if applicable.

17. Comment record — planned fields

Field	Type / example	Purpose
id	unique ID	Identifies the comment.
ticketId	ticket ID	Ticket this comment belongs to.
authorId	user ID	Authenticated user who wrote it.
body	Text	The message content.
createdAt	ISO timestamp	When the message was posted.

18. Relationships

- One customer can create many tickets.
- One support agent can be assigned many tickets.
- One ticket belongs to one customer.
- One ticket may have zero or one assigned agent at a time.
- One ticket can have many comments.
- One user can write many comments across tickets they are allowed to access.

```
USER (1) -------- (many) TICKETS
  |                       |
  |                       |
  +---- (many) COMMENTS --+
            (each comment has one author
             and belongs to one ticket)
```

19. Example temporary records

```
const users = [
  {
    id: "u1",
    name: "Slim",
    email: "slim@example.com",
    role: "customer",
    accountStatus: "active"
  },
  {
    id: "u2",
    name: "Sarah",
    email: "sarah@techworld.example",
    role: "support_agent",
    accountStatus: "active"
  }
];

const tickets = [
  {
    id: "t1024",
    title: "Laptop Wi-Fi connection problem",
    description: "My laptop cannot connect to my home Wi-Fi.",
    customerId: "u1",
    assignedAgentId: "u2",
    status: "in_progress",
    priority: "high",
    createdAt: "2026-11-10T09:00:00.000Z"
  }
];

const comments = [
  {
    id: "c1",
    ticketId: "t1024",
    authorId: "u2",
    body: "Can other devices connect to the same network?",
    createdAt: "2026-11-10T09:30:00.000Z"
  }
];
```

The sample data is illustrative. Use safe fake credentials only. Never put real customer information or passwords in test fixtures or public repositories.

E. API specification

20. API conventions

- Prefix application endpoints with /api.
- Use JSON request and response bodies for the API.
- Use plural resource names such as /users and /tickets.
- Use route parameters for a specific resource, such as /tickets/:ticketId.
- Use query parameters for filtering, sorting and pagination, such as /tickets?status=open&page;=1.
- Use POST to create a resource, GET to read, PATCH for partial updates, PUT for full replacement when appropriate, and DELETE to remove a resource.
- Use consistent HTTP status codes and a consistent error response shape.
- Do not expose password hashes or secrets in API responses.

21. Planned endpoint catalogue

Method	Endpoint	Who can use it	Purpose
GET	/api/health	Public	API status, timestamp and uptime.
POST	/api/auth/register	Public	Create a customer account only.
POST	/api/auth/login	Public	Authenticate an existing account.
GET	/api/users	Admin	List users; later support filters and pagination.
GET	/api/users/:userId	Admin; self-view may be added	Retrieve user details within permission rules.
PATCH	/api/users/:userId	Admin; limited self-profile route may be added	Update permitted user fields.
PATCH	/api/users/:userId/status	Admin	Enable or disable an account.
POST	/api/agents	Admin	Create/provision a support-agent account.
GET	/api/tickets	Customer, agent, admin	Return only tickets visible to the authenticated role.
POST	/api/tickets	Customer	Create a support ticket.
GET	/api/tickets/:ticketId	Authorized ticket participants/admin	Retrieve ticket details.
PATCH	/api/tickets/:ticketId	Authorized agent/admin	Update permitted ticket fields.
PATCH	/api/tickets/:ticketId/assignment	Admin	Assign or unassign an agent.
PATCH	/api/tickets/:ticketId/status	Authorized agent/admin	Change status using valid transitions.
GET	/api/tickets/:ticketId/comments	Authorized ticket participants/admin	Retrieve conversation.
POST	/api/tickets/:ticketId/comments	Authorized ticket participants	Add a comment.

22. Health endpoint contract

Method: GET. Path: /api/health. Access: public. Purpose: confirm the server is running and return basic diagnostic information.

Example response shape

```
{
  "status": "ok",
  "timestamp": "2026-11-10T10:30:00.000Z",
  "uptime": 125.42
}
```

23. Registration contract

Method: POST. Path: /api/auth/register. Access: public. The request accepts a name, email and password. The server validates them, ensures the email is not already in use, hashes the password in the authentication phase, assigns the customer role, and returns safe account information. It must not accept a user-selected role.

```
{
  "name": "Slim",
  "email": "slim@example.com",
  "password": "example-only-password"
}
```

24. Create-ticket contract

Method: POST. Path: /api/tickets. Access: authenticated customer. The customer provides a title and description, and may optionally provide a category. The server derives customerId from the authenticated identity, generates the ticket ID, sets status to open, sets a default priority if omitted, and starts with no assigned agent unless the workflow is changed.

```
{
  "title": "Laptop Wi-Fi connection problem",
  "description": "My laptop cannot connect to my home Wi-Fi.",
  "category": "technical_support"
}
```

25. List-ticket query examples

```
GET /api/tickets?status=open
GET /api/tickets?priority=high
GET /api/tickets?status=in_progress&priority=urgent
GET /api/tickets?page=1&limit=10
GET /api/tickets?sort=createdAt&order=desc
```

The server must still apply role-based visibility. A customer asking for /api/tickets must receive their own tickets, not every customer's tickets. An agent receives only the queue and assigned records they are authorized to see. Admins can view all tickets.

26. Standard response and error conventions

Successful responses should return the requested data and an appropriate status code. Errors should be consistent and should not expose stack traces, secrets or internal implementation details to clients.

Status code	Typical use
200 OK	A read or update succeeded.
201 Created	A new resource was created.
204 No Content	A deletion or action succeeded with no response body.
400 Bad Request	The request is malformed or fails validation.
401 Unauthorized	The user is not authenticated or the credentials/token are invalid.
403 Forbidden	The user is authenticated but lacks permission.
404 Not Found	The requested resource does not exist or is not visible to the user.
409 Conflict	A conflict such as a duplicate email.
500 Internal Server Error	Unexpected server-side failure.

F. Backend architecture and complete file plan

27. Target folder structure

This is the intended structure once the relevant phases are reached. It is a destination, not a checklist to create every file immediately. Some filenames may be adjusted as the implementation becomes clearer.

```
supporthub/
■■■ src/
■   ■■■ app.js
■   ■■■ config/
■   ■   ■■■ env.js
■   ■■■ routes/
■   ■   ■■■ index.js
■   ■   ■■■ health.routes.js
■   ■   ■■■ auth.routes.js
■   ■   ■■■ users.routes.js
■   ■   ■■■ agents.routes.js
■   ■   ■■■ tickets.routes.js
■   ■■■ controllers/
■   ■   ■■■ health.controller.js
■   ■   ■■■ auth.controller.js
■   ■   ■■■ users.controller.js
■   ■   ■■■ agents.controller.js
■   ■   ■■■ tickets.controller.js
■   ■■■ services/
■   ■   ■■■ auth.service.js
■   ■   ■■■ users.service.js
■   ■   ■■■ agents.service.js
■   ■   ■■■ tickets.service.js
■   ■■■ middleware/
■   ■   ■■■ authenticate.js
■   ■   ■■■ authorize.js
■   ■   ■■■ validate.js
■   ■   ■■■ not-found.js
■   ■   ■■■ error-handler.js
■   ■■■ data/
■   ■   ■■■ users.js
■   ■   ■■■ tickets.js
■   ■   ■■■ comments.js
■   ■■■ validators/
■   ■   ■■■ auth.validators.js
■   ■   ■■■ users.validators.js
■   ■   ■■■ tickets.validators.js
■   ■■■ utils/
■   ■   ■■■ create-id.js
■   ■   ■■■ api-error.js
■   ■■■ models/                (MongoDB phase)
■       ■■■ user.model.js
■       ■■■ ticket.model.js
■       ■■■ comment.model.js
■■■ tests/
■   ■■■ health.test.js
■   ■■■ auth.test.js
■   ■■■ users.test.js
■   ■■■ tickets.test.js
■■■ .env
■■■ .env.example
■■■ .gitignore
■■■ package.json
■■■ package-lock.json
■■■ server.js
```

28. What each part is responsible for

Folder/file	Responsibility and examples
src/app.js	Creates/configures the Express app, registers shared middleware and mounts route modules. It should not need to start listening itself.
server.js	Loads environment configuration, imports the app and starts the HTTP server on the configured port.
config/	Application configuration, such as reading and validating required environment variables.
routes/	Declares HTTP methods and paths, then connects each route to middleware and controller functions.
controllers/	Handles HTTP concerns: reads params/body/query/user identity, calls services, chooses status code and sends response.
services/	Contains business rules and reusable application operations, such as creating a ticket or assigning an agent.
middleware/	Functions in the request-response pipeline: authentication, authorization, validation, not-found and error handling.
data/	Temporary arrays/objects for the pre-database phases. Later replaced by persistent database operations.
validators/	Rules describing acceptable input for registration, user updates and ticket operations.
utils/	Small generic helpers, such as ID creation or a reusable API error class. Avoid turning this into a miscellaneous dumping ground.
models/	MongoDB/Mongoose schemas and models, introduced only in the database phase.
tests/	Automated checks for endpoint behaviour, permissions, validation and important business rules.
.env	Local configuration values; never commit this file if it contains secrets.
.env.example	Names of required environment variables with safe placeholder values, never real secrets.
.gitignore	Excludes node_modules, .env, logs and other generated/local files from Git.
package.json	Project metadata, scripts and dependency declarations.

29. What routes, controllers, services and middleware mean together

Suppose a customer sends POST /api/tickets. The route matches the URL and method. Authentication middleware checks who is making the request. Validation middleware checks the supplied title and description. The controller handles the HTTP request and calls a ticket service. The service applies business rules and stores the new ticket in temporary data or the database. The controller sends the response.

Not every route needs every layer. A small health endpoint may be handled by a simple route handler. We introduce separation when it makes the code easier to understand, test and maintain.

30. Suggested responsibility boundaries

- Routes should stay declarative; avoid putting all business logic inside route files.
- Controllers should not become large collections of data manipulation rules.
- Services should not depend on Express response objects; they should focus on application operations.
- Middleware should do one reusable request-pipeline job and call next() or finish the response appropriately.
- Data modules should contain temporary records, not route handlers.
- Utilities should be generic helpers, not a second services folder.
- Avoid premature abstraction: if a feature is tiny, begin simply and refactor when there is a clear benefit.

G. React + Vite frontend specification

31. Frontend goal

After the backend is functional and its API rules are understood, build a React + Vite frontend that consumes the API. The frontend is the user interface; it is not the source of truth for roles, permissions or data validation.

32. Customer experience

Page/view	Required behaviour
Register	Collect name, email and password; show validation and success/error feedback. No role picker.
Login	Authenticate the user and route them to the customer area.
Customer dashboard	Show a summary of the customer's tickets and their current statuses.
Create ticket	Form for title, description and optional category; show field errors.
My tickets	List only the signed-in customer's tickets; support basic status filtering.
Ticket details	Show description, status, priority, assigned agent and conversation history; allow replies.
Profile	Show basic account information; allow only permitted profile changes.

33. Support-agent experience

Page/view	Required behaviour
Agent dashboard	Show assigned tickets and, if enabled, an unassigned/team queue.
Ticket queue	Filter by status, priority and assignment.
Ticket details	Show customer request and conversation; allow agent replies.
Ticket actions	Change status and priority; assignment is admin-controlled in the first version.
Agent profile	Show agent account information.

34. Administrator experience

Page/view	Required behaviour
Admin dashboard	Show high-level counts: total, open, in progress and resolved tickets.
All tickets	Search/filter across the system and open ticket details.
User management	View users and disable/enable accounts where appropriate.
Agent management	Create/provision agent accounts and view agent records.
Assignment management	Assign an active support agent to a ticket or remove an assignment.

35. Frontend navigation and access

- After login, the app reads the authenticated user's role from trusted API/session data.
- Customers are directed to the customer dashboard; agents to the agent workspace; admins to the admin area.
- Protect frontend routes for usability, but remember that frontend route guards are not security. The API must independently enforce permissions.
- Show loading, empty, success and error states rather than leaving the user uncertain.
- Use responsive layouts so the dashboards work on laptop and mobile screens.
- Keep API communication in a reusable client/service layer rather than scattering fetch logic through every component.

36. Suggested frontend structure (later)

```
supporthub-client/
■■■ src/
■   ■■■ app/
■   ■   ■■■ App.jsx
■   ■   ■■■ router.jsx
■   ■■■ components/
■   ■   ■■■ layout/
■   ■   ■■■ forms/
■   ■   ■■■ tickets/
■   ■■■ pages/
■   ■   ■■■ auth/
■   ■   ■■■ customer/
■   ■   ■■■ agent/
■   ■   ■■■ admin/
■   ■■■ context/ or store/
■   ■■■ services/
■   ■   ■■■ api.js
■   ■   ■■■ auth.service.js
■   ■   ■■■ tickets.service.js
■   ■■■ hooks/
■   ■■■ styles/
■■■ .env
■■■ package.json
■■■ vite.config.js
```

H. Development roadmap and learning objectives

37. Phase 1 — Project foundation

Build: Initialize the Node project, install Express, configure the server, create a health endpoint, add a basic route structure, initialize Git and push to GitHub.

Research: Node.js modules, npm, package.json, Express app creation, listening on a port, environment variables, request/response objects, HTTP methods, JSON responses and Git basics.

Complete when: The server starts reliably; GET /api/health returns status, timestamp and uptime; the endpoint is tested; the repository is connected to GitHub; .env and node_modules are excluded from commits.

38. Phase 2 — User management (temporary data)

Build: Create user records for local testing; list users; retrieve one by ID; update allowed fields; delete or disable a user; add basic filtering.

Research: CRUD, route parameters, query parameters, request body parsing, array methods, ID generation, status codes, validation and controller separation.

Complete when: Each operation works for valid input; missing IDs are handled; duplicate email is rejected; sensitive fields are never returned; admin-only operations are protected when authentication exists.

39. Phase 3 — Ticket management

Build: Create tickets; list tickets; retrieve one; update title/description where allowed; update status and priority; assign an agent; filter by status and priority.

Research: Resource design, CRUD, business rules, ownership, relationships using IDs, filtering, PUT vs PATCH, validation and service functions.

Complete when: New tickets receive an ID, customer ownership, default status and timestamps; invalid status/priority values are rejected; customers cannot create tickets for another user.

40. Phase 4 — Comments and conversations

Build: Add comments to tickets and retrieve a ticket's conversation in chronological order.

Research: Nested routes, relationships, chronological sorting, author identity, input validation and access checks.

Complete when: Comments belong to a real ticket; author identity comes from the authenticated user; unauthorized users cannot read or write the conversation.

41. Phase 5 — Middleware and validation

Build: Reusable request validation, not-found handling, centralized error handling, request logging and any shared request checks.

Research: Express middleware signature, next(), middleware order, error middleware, validation libraries or custom validation, error classes and consistent response shapes.

Complete when: Invalid requests return useful client errors; unknown routes return 404; unexpected errors are handled centrally; sensitive internal details are not leaked.

42. Phase 6 — Authentication and authorization

Build: Customer registration and login; secure password hashing; authenticated requests; role-based permissions; admin-created agent accounts; controlled initial admin setup.

Research: Authentication vs authorization, password hashing, tokens or sessions, HTTP authorization headers, authentication middleware, role checks, secret management and account states.

Complete when: Customers can register and log in; elevated roles cannot be self-assigned; protected endpoints reject anonymous requests; each role can perform only permitted actions.

43. Phase 7 — Professional API features

Build: Pagination, sorting, searching, richer filters, request logs, automated tests and API documentation.

Research: Pagination strategies, query parsing, testing Express apps, integration tests, test data, API documentation and logging practices.

Complete when: Large lists are paginated; filters combine correctly; important success and failure cases have tests; another developer can understand how to call the API.

44. Phase 8 — MongoDB integration

Build: Replace in-memory arrays with persistent MongoDB records and Mongoose models; preserve the existing API behaviour.

Research: MongoDB documents/collections, Mongoose schemas/models, references, CRUD operations, indexes, connection handling, environment configuration and database errors.

Complete when: Data survives server restarts; user/ticket/comment relationships work; validation and permission rules remain enforced; database failures are handled.

I. Quality, testing and security

45. General validation rules

- Trim strings where appropriate and reject values that are empty after trimming.
- Validate email format and enforce uniqueness.
- Set reasonable maximum lengths for names, titles, descriptions and comments.
- Validate status, priority and role against allowed values.
- Do not accept IDs, roles, author IDs or timestamps from the client when the server should generate or derive them.
- Return field-level validation messages where practical.
- Validate that referenced users and tickets exist before linking records.

46. Security requirements

- Never store plain-text passwords; hash them with a suitable password-hashing library in the authentication phase.
- Never expose password hashes, secrets or authentication tokens in normal user responses.
- Never trust role, customerId, authorId or assignedAgentId simply because the client sent them.
- Enforce authentication and authorization on the server for every protected endpoint.
- Prevent insecure direct object reference: knowing another ticket ID must not let a customer read that ticket.
- Keep environment secrets out of source control and use safe environment configuration in deployment.
- Use HTTPS in production and configure browser/API access deliberately.
- Avoid detailed stack traces or internal errors in production responses.
- Apply sensible request-size limits and consider rate limiting for public authentication endpoints before production.

47. Manual test scenarios

Scenario	Expected result
GET health endpoint	Returns a successful status, timestamp and uptime.
Register with valid customer details	Creates a customer account.
Register with role=admin	Does not create an admin account; role is ignored or request is rejected.
Register using an existing email	Returns a conflict/validation response.
Create ticket while unauthenticated	Returns 401 once authentication is enabled.
Customer creates ticket	Ticket is linked to the signed-in customer and starts open.
Customer requests another customer's ticket	Access is denied or the resource is not exposed.
Assign a non-agent account	Rejected by the backend.
Agent updates an allowed ticket	Ticket changes and updatedAt is refreshed.
Submit unsupported status	Rejected with a validation error.
Post a comment to a missing ticket	Returns not found.
Unauthorized user reads comments	Access is denied.
Unknown API path	Returns a consistent 404 response.
Unexpected server error	Returns a safe generic error without leaking internals.

48. Test levels

Level	What it checks
Manual API checks	Use Postman or a similar client to inspect requests and responses while learning.
Unit tests	Test isolated helper functions and business rules.
Integration tests	Test endpoints with middleware, services and test data working together.
End-to-end tests (later)	Test a complete user journey through the frontend and backend.

J. GitHub, collaboration and delivery

49. Suggested Git workflow

- Create a Git repository for the backend project named supporthub (or a clear equivalent).
- Commit the initial project setup before adding many features.
- Use clear commit messages describing a focused change, for example: 'Add health check endpoint'.
- For a solo learning project, working on main is acceptable initially. As the project grows, practise feature branches and pull requests.
- Keep the README updated with project purpose, setup instructions, environment variables and available scripts.
- Never commit node_modules, .env, real credentials, private customer data or generated secrets.

50. Initial .gitignore expectations

```
node_modules/
.env
.env.*
!.env.example
coverage/
logs/
*.log
.DS_Store
```

51. README contents

- Project name and short description.
- What problem SupportHub solves.
- Technology stack.
- Prerequisites and installation instructions.
- How to configure environment variables using .env.example.
- How to start the development server.
- Available API endpoints or link to API documentation.
- Current implementation status and known limitations.

52. Deployment readiness checklist

- Application starts from a documented command and uses the deployment platform's port configuration.
- Secrets are stored in the hosting environment, not in the repository.
- Database connection uses production configuration and does not rely on temporary in-memory data.
- CORS and allowed frontend origins are configured deliberately.
- HTTPS is enabled by the hosting platform.
- Errors are logged safely; private information is not written to logs.
- Health endpoint can be used by a hosting health check.
- Core API tests pass against the release candidate.
- README and environment variable documentation are current.

53. Definition of done for a feature

- The feature matches the product behaviour described in this blueprint.
- The correct user role can perform the action.
- Users without permission cannot perform the action by calling the API directly.
- Valid input succeeds and invalid input is handled clearly.
- Missing resources and unexpected failures are handled.

- The response uses an appropriate status code and does not expose sensitive data.
- The implementation is understandable and has been reviewed.
- Relevant tests or documented manual checks have been completed.

K. Project decisions and future enhancements

54. Confirmed decisions for the first version

- The project is a customer support ticket management system, not event ticketing.
- The fictional example company is TechWorld; the app itself is reusable by a company with a support team.
- There are three roles: customer, support_agent and admin.
- Public registration creates customers only.
- An administrator provisions support agents; public users cannot self-select the agent role.
- The first administrator is created through controlled setup, not public registration.
- The first backend implementation uses JavaScript and Express.
- Temporary in-memory data is used before MongoDB is introduced.
- The backend is built and understood before the React + Vite frontend.
- The project is developed in phases with research and code review between milestones.

55. Possible future enhancements — not required initially

- Email notifications when a ticket is created, assigned, updated or resolved.
- Agent invitations by email and password setup links.
- Internal notes visible only to agents and admins.
- File attachments and screenshots.
- Ticket categories, tags and saved filters.
- Service-level targets and response-time reporting.
- Agent workload and performance dashboards.
- Customer satisfaction ratings after resolution.
- Audit history showing who changed ticket status, priority or assignment.
- Multiple companies/tenants using separate workspaces.
- Live updates using WebSockets or server-sent events.

56. What is deliberately out of scope at the start

- Building a real laptop diagnostic or remote repair tool.
- Payment processing or e-commerce checkout.
- Public event/concert ticket sales.
- Complex multi-company tenancy.
- Live chat before the basic ticket/comment workflow is reliable.
- File uploads before the security and storage design is understood.
- Microservices or distributed infrastructure; a well-structured Express application is sufficient.

L. Quick reference and first milestone

57. Quick answers

Question	Answer
What is a ticket?	A stored record of a customer's problem or request.
Who creates tickets?	Customers in the initial version.
Who works on tickets?	Support agents.
Who manages agents and assignments?	Administrators.
Can a customer choose to become an agent?	No. An administrator authorizes agent accounts.
Can a public user register as admin?	No. The initial admin is provisioned securely.
Does SupportHub repair the laptop?	No. It manages the request and communication around the support work.
Why use temporary arrays first?	To learn Express request handling and business logic before database complexity.
When do we use MongoDB?	After the core Express phases, in Phase 8.
When do we build React?	After the backend API is understood and working.

58. Your first assignment — Phase 1, Task 1

Do not build the entire application yet. Begin with the smallest working foundation.

- Create a local directory named `supporthub`.
- Initialize the Node.js project.
- Install Express.
- Create the initial server entry point and Express app.
- Implement GET /api/health.
- Return an API status, a current timestamp and server uptime.
- Test the endpoint with Postman or another HTTP client.
- Initialize Git, create a GitHub repository and push the initial project.
- Ensure .env and node_modules are not committed.

What to bring for review

- Your folder/file structure.
- Your package.json and relevant source files.
- The health endpoint request and response.
- Your GitHub repository link, if it is public (do not share secrets).
- Any errors or questions you encountered.

Do not move to user management until the health endpoint and project setup are understood and reviewed.

59. Working agreement

Your responsibility: Research the concepts needed for the current task, implement the feature yourself, test it, and be ready to explain your decisions.

My responsibility: Clarify product requirements, help you identify what to research, review your code, explain mistakes, suggest improvements and provide hints before full solutions when possible.

Our rule: The blueprint answers “what should the application do?” Your research answers “how do I implement it?” We move forward when the feature works and you understand it.

SupportHub is a learning project, but we will treat its requirements, security boundaries and code quality seriously. Build it one understandable feature at a time.